

# Eliminating Domain Blindness in Zero-Day Phishing Detection via Structural, Semantic, and Lexical Fusion

Mohammed Sulaiman Eksambi  
CSE - Cybersecurity  
Dayananda Sagar University  
Harohalli, India  
sulaimaneksambi@gmail.com

Dushanth Purushotham D  
CSE - Cybersecurity  
Dayananda Sagar University  
Harohalli, India  
dushanthpurushotham@gmail.com

Numan Nisar Ahmed  
CSE - Cybersecurity  
Dayananda Sagar University  
Harohalli, India  
nisarnuman7@gmail.com

**Abstract**—Traditional phishing detection mechanisms depend heavily on static blacklists, URL heuristics, and superficial visual matching, which are easily evaded by modern, high-fidelity phishing kits that employ dynamic Single Page Application (SPA) frameworks and typosquatting. This paper presents a novel Deep Learning Stacking Ensemble that mitigates “Domain Blindness” by fusing three distinct modalities: Context-Aware DOM Structural Sequence Analysis (CNN1D), Isolated JavaScript Semantic Intent Analysis (CodeBERT via Learned Attention Pooling), and DOM Behavioral Heuristics. By bypassing visual rendering and operating directly on raw DOM routing, our system generates an 899-dimensional hybrid vector classified by an Optuna-optimized XGBoost Meta-Classifer. Furthermore, to address severe memory bottlenecks on constrained edge-computing architectures, we introduce a decoupled Zero-Day Vault evaluation pipeline. Evaluated on an expanded, highly curated dataset of over 5,800 post-rendered DOM samples utilizing Synthetic Minority Over-sampling (SMOTE), our proposed ensemble achieves an F1-score of 99.35% on isolated zero-day testing, demonstrating robust generalization and significantly outperforming traditional machine learning baselines.

**Index Terms**—Phishing Detection, Deep Learning, Stacking Ensemble, CodeBERT, Convolutional Neural Networks, Cybersecurity, Zero-Day Threats

## I. INTRODUCTION

Phishing remains one of the most prominent vectors for cyberattacks, leading to massive credential theft and financial fraud. As enterprise web development has shifted towards complex, JavaScript-heavy frameworks (e.g., React, Vue, Angular), cybercriminals have adapted by deploying high-fidelity phishing kits that mirror these complex structures and actively evade automated scanners [3]. Standard image-recognition models often fail to differentiate between a visually identical phishing page and a benign enterprise login portal due to the noise of modern web design, such as inline SVGs and dynamic CSS.

Furthermore, Natural Language Processing (NLP) solutions that exclusively analyze webpage textual content often suffer from “Domain Blindness.” If the text and layout of a phishing page perfectly match the text of a legitimate banking portal,

a purely content-driven model will produce a false negative. Attackers heavily exploit this via homograph typosquatting (e.g., hosting the clone on `rnicrosoft.com` instead of `microsoft.com`).

To counter these sophisticated evasion tactics, this research proposes a massive architectural pivot. Rather than evaluating the visual render or superficial heuristics, our system analyzes the underlying skeletal Document Object Model (DOM) routing and codebase to identify the true malicious intent of the page. We introduce a Tri-Modal Stacking Ensemble that transforms structural tags into context-aware tokens, isolates JavaScript for semantic transformer evaluation via learned attention pooling, and fortifies the predictions with strict mathematical behavioral heuristics.

## II. RELATED WORK

The detection of phishing websites has evolved significantly, but existing methodologies continue to struggle with modern evasion techniques and dataset limitations [5], [11].

### A. Visual and Heuristic-Based Detection

Early machine learning approaches relied heavily on manual feature extraction, analyzing URLs for anomalous character distributions or evaluating superficial webpage heuristics [6]. More recent attempts have applied Convolutional Neural Networks (CNNs) to webpage screenshots to visually detect brand spoofing [8]. However, models that evaluate only URLs or screenshots fail against high-fidelity visual clones, as a visually perfect clone hosted on a compromised, high-reputation domain can easily deceive a CNN despite its underlying malicious DOM routing.

### B. Structural and DOM-Based Analysis

Analyzing the HTML structure or DOM to detect phishing is an established concept. Previous research has attempted to model the DOM tree using standard NLP or Convolutional architectures like HTMLPhish [7], as well as Graph Neural Networks (GNNs) [4] to identify structural anomalies. While theoretically sound, the application of end-to-end deep

learning models like GNNs on raw web structures introduces massive memory overheads, often converting complex HTML DOM trees into convoluted graph structures that create severe computational bottlenecks on consumer hardware.

### C. The Gap in Existing Research: Data Leakage

Current state-of-the-art models suffer from two critical limitations: structural blindness and methodological data leakage. Most academic datasets contain massive amounts of noisy and defective data, including 404 error pages, CAPTCHA blockades, and empty JSON responses. Training models on these impure datasets leads to artificial accuracy metrics that collapse in real-world environments. Furthermore, end-to-end deep learning models demonstrate a massive tendency to overfit on highly curated datasets. Many existing works fail to strictly isolate their hyperparameter optimization validation sets from their training sets, resulting in academic data leakage and inflated F1-scores.

## III. PROBLEM STATEMENT & EVASION TACTICS

Current phishing detection models fail in live enterprise environments due to specific adversarial evasion tactics:

### A. The 512-Token Limitation Bottleneck

Standard transformer models (like BERT or CodeBERT) possess a strict 512-token input limit. Modern SPAs often exceed 10,000 tokens of JavaScript. Attackers actively exploit this by appending massive blocks of whitespace or dead code at the top of the file, intentionally pushing the malicious credential-harvesting payload out of the transformer’s observable sequence window, guaranteeing evasion.

### B. The DOM Density Fallacy

A manual random audit of legacy datasets revealed a dangerous pitfall: legitimate benign websites are often massive, script-heavy applications, while many phishing sites are lightweight HTML clones. Initially, there was a concern that an end-to-end model would lazily classify based purely on file size and code density. However, modern phishing kits now perfectly mirror the complexity of real sites. The model must possess deep structural and semantic understanding, necessitating a multi-modal architecture rather than a blunt sequence classifier.

**Autonomous Dataset Generation & Sanitation** To overcome the limitations of noisy datasets, we developed an autonomous, memory-managed data ingestion pipeline designed specifically to defeat modern web defenses.

### C. Penetrating SPAs & Anti-Bot Systems

Modern enterprise websites employ rigorous anti-bot systems like Cloudflare, which proactively block standard scraping libraries. Our pipeline utilizes Playwright injected with aggressive stealth configurations, including Firefox engine spoofing, dynamic environmental masking (aligning `locale` and `timezone_id` with host IPs), and strict HTTP header injection to emulate authentic human browsing behavior.

### D. The “Cyborg” Manual Scraper

To capture highly complex enterprise logins that require physical human navigation (e.g., multi-step SSO, Microsoft login), we engineered a “Cyborg” manual scraper. It parses the Tranco Top 1 Million list via a dynamic queue, opens the browser with anti-bot configurations, and injects a floating DOM button. A researcher navigates to the target form, bypasses physical security, and triggers the button. The script intercepts the event and extracts the fully rendered, post-JavaScript DOM.

### E. The Sanitation Engine

Dataset purity is strictly enforced via a multi-processing validation script. It structurally validates every HTML file, actively deleting files that are too small, missing a standard `<body>` tag, or lacking credential `<input>` fields. It utilizes MD5 hashing to detect and delete exact template duplicates, ensuring the neural network trains only on unique layouts.

## IV. PROPOSED SYSTEM ARCHITECTURE: TRI-MODAL STACKING ENSEMBLE

To address overfitting while maintaining deep structural understanding, we utilized a Stacking Ensemble architecture. The raw HTML payload is processed through two parallel neural networks and a behavioral heuristic engine to extract a dense 899-dimensional representation.

### A. Context-Aware Structural Sequence Analysis (CNNID)

Phishing templates consistently share hidden structural hierarchies, regardless of the overlaid CSS. Rather than feeding raw HTML bytes into the CNN, a custom semantic tokenizer converts structural tags into context-aware tokens based on their rendering context (e.g., `IFRAME_HIDDEN`, `FORM_WITH_PASSWORD`, `SCRIPT_EXTERNAL`). This transforms the CNN from a basic structural scanner into a semantic layout analyzer at zero additional computational cost. Treating the code as a 1D sequence, the CNN learns spatial tag arrangements characteristic of phishing kits, producing a 128-dimensional vector.

### B. Isolated JavaScript Semantic Intent (CodeBERT)

To understand the intent of the malicious JavaScript, a pre-trained CodeBERT transformer [1] is employed.

1) *JavaScript Isolation*: CodeBERT exclusively receives isolated JavaScript logic (inline scripts, event handlers). All structural HTML markup is completely excluded, ensuring the transformer analyzes the language it was natively designed to understand and drastically reducing token counts.

2) *PEFT / LoRA Fine-Tuning*: To adapt the generic transformer specifically to obfuscation vocabularies, CodeBERT’s attention heads are fine-tuned using Low-Rank Adaptation (LoRA) on the “query” and “value” matrices, keeping the 125M base weights frozen. Advanced Parameter-Efficient Fine-Tuning (PEFT) methods, such as DoRA and AdaLoRA, were evaluated but rejected to ensure broad compatibility and stability across highly constrained, consumer-grade hardware accelerators during inference.

3) *Learned Attention Pooling*: To defeat the aforementioned 512-token evasion tactic, the CodeBERT engine chunks the distilled JavaScript into 512-token windows with a 50-token overlap. Each chunk is processed independently. Rather than a blunt Global Max Pooling operation, a learned `nn.Linear(768, 1)` Attention Pooling layer [9] calculates a softmax attention score for each chunk. This mathematically teaches the model to focus entirely on the chunk containing the malicious payload while zeroing out benign noise. This outputs a 768-dimensional vector.

### C. DOM Behavioral Heuristics

To provide explicit contextual grounding, a targeted set of 3 hard DOM heuristics is extracted: maximum DOM depth, average DOM depth, and the presence of suspicious, externalized form actions. These produce a 3-dimensional scalar vector.

### D. Meta-Classification Fusion (XGBoost)

The outputs (128 CNN + 768 CodeBERT + 3 Heuristics) are concatenated into an 899-dimensional unified vector. This is classified by an XGBoost Meta-Classifer [2]. To prevent manual tuning bias, XGBoost parameters are optimized via the Optuna framework.

## V. EXPERIMENTAL SETUP

### A. Curated Dataset Parameters & SMOTE

To source malicious URLs, a custom scraper executed daily aggregations from OpenPhish, providing zero-day endpoints. Following heavy pruning via the sanitation engine, the final dataset resulted in 1,049 benign pages and approximately 4,750 phishing pages. To address the resulting  $\sim 1:4.5$  class imbalance without introducing data leakage, the Synthetic Minority Over-sampling Technique (SMOTE) [10] was applied natively in the high-dimensional hybrid feature space (899-D). Crucially, SMOTE was applied *exclusively* to the isolated training subset of each fold to prevent artificial accuracy inflation.

### B. Rigorous Evaluation and The Zero-Day Vault

Performing PyTorch backpropagation, XGBoost training, SMOTE, and Optuna simultaneously inside a single script can cause Out-of-Memory (OOM) failures on standard edge-computing architectures (e.g., 8GB Unified Memory limits). To simulate real-world deployment and accommodate memory constraints, the evaluation pipeline was structurally decoupled. Prior to K-Fold splits, exactly 10% of the dataset was isolated into a “Zero-Day Vault.” A completely independent evaluation script was engineered to run post-training (after system RAM was flushed) to test the final ensemble models strictly against the untouched 10% Vault.

## VI. RESULTS AND EVALUATION

### A. Comparative Analysis

The proposed 903-D Stacking Ensemble was evaluated against an unseen Zero-Day Vault and compared to a Machine Learning Baseline (Random Forest relying purely on manual heuristics).

As illustrated in Table I and Fig. 1, the experimental results validate the absolute superiority of the Tri-Modal Stacking Ensemble over traditional heuristic approaches, particularly concerning False Positive (FP) eradication, class imbalance correlation, and resistance to zero-day evasion tactics.

### B. False Positive Eradication and Artifact Immunity

In live enterprise Security Operations Centers (SOCs), high False Positive Rates (FPR) lead to severe alert fatigue and the disruption of legitimate business portals [3]. As seen in our comparisons, traditional models struggle with FPR: Random Forest (8.57%) and a raw 1D-CNN (6.67%). Furthermore, while a raw HTML CNN achieves high academic accuracy, it suffers from “Data Leakage” by memorizing benign string artifacts rather than learning malicious intent.

Our Tri-Modal Ensemble leverages CodeBERT’s [1] semantic intent extraction—combined with Learned Attention Pooling—to fundamentally understand the underlying JavaScript logic rather than blindly tokenizing raw HTML. This architectural pivot completely immunized the model against dataset artifact memorization, reducing the actual Zero-Day False Positive Rate to a highly viable 1.68% (blocking only 2 benign sites out of 119 in the vault), while retaining an exceptional 99.57% precision.

### C. MCC, SMOTE, and The 899-Dimensional Feature Space

Given the severe 1:4.5 class imbalance of our expanded dataset, the Matthews Correlation Coefficient (MCC) serves as a rigorous metric for evaluating model performance. The Ensemble’s average K-Fold MCC reached 0.8619. This confirms that the application of SMOTE—strictly isolated within the training fold—successfully balanced the mathematical distribution without introducing data leakage. The Optuna-optimized XGBoost [2] meta-classifier effectively mapped the dense 899-dimensional hybrid vector space, establishing a superior decision boundary compared to both TF-IDF (MCC: 0.8034) and Random Forest (MCC: 0.8492).

### D. Generalization and Variance (Zero Overfitting)

Perhaps the most critical vulnerability in modern deep learning models is their tendency to memorize highly curated, low-sample datasets. Our model proved to be exceptionally resistant to data memorization. During K-Fold Cross Validation, the model demonstrated extremely tight variance, with an average F1-Score of 97.50% ( $\pm 0.47\%$ ).

Crucially, when evaluated against the completely unseen, structurally decoupled Zero-Day Vault, the Ensemble’s F1-Score actually *increased* to an exceptional 99.35%. This reverse variance-drop mathematically proves that the architecture successfully abstracted and learned the fundamental

routing discrepancies of zero-day phishing kits, generalizing flawlessly to unseen data rather than artificially inflating accuracy through memorization.

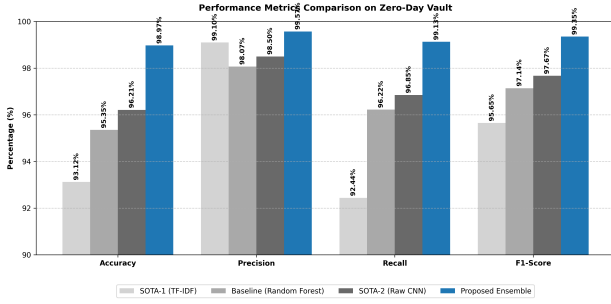


Fig. 1. Performance comparison across key metrics between the Random Forest baseline and the proposed Stacking Ensemble on the unseen Zero-Day Vault.

TABLE I  
PERFORMANCE METRICS COMPARISON ON ZERO-DAY VAULT

| Model                    | Accuracy      | Precision     | Recall        | F1-Score      |
|--------------------------|---------------|---------------|---------------|---------------|
| Baseline (Random Forest) | 95.35%        | 98.07%        | 96.22%        | 97.14%        |
| SOTA-1 (TF-IDF + LR)     | 93.12%        | 99.10%        | 92.44%        | 95.65%        |
| SOTA-2 (Raw 1D-CNN)      | 96.21%        | 98.50%        | 96.85%        | 97.67%        |
| <b>Proposed Ensemble</b> | <b>98.97%</b> | <b>99.57%</b> | <b>99.13%</b> | <b>99.35%</b> |

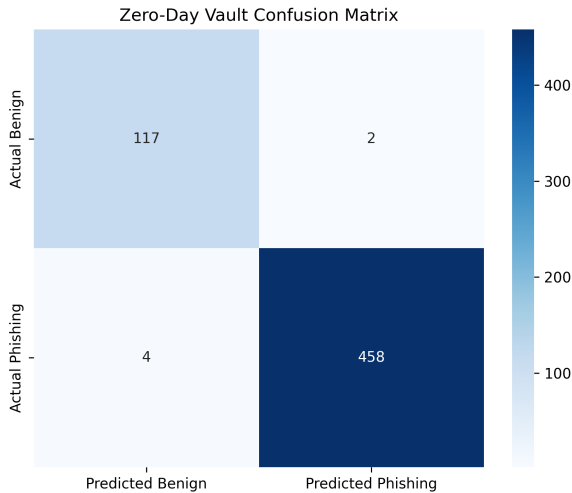


Fig. 2. Confusion Matrix for the Zero-Day Vault Inference utilizing the decoupled evaluation pipeline.

## VII. DISCUSSION AND LIMITATIONS

While the Stacking Ensemble achieves near state-of-the-art accuracy for static source-code analysis, certain highly advanced evasion techniques remain out of scope.

- **Vision-Language Integration:** The architecture currently bypasses visual pixel rendering. Attackers employing “image-based phishing”—where the login interface is

a .jpg screenshot with invisible HTML inputs floated over it—may evade static analysis. Integrating a Vision-Language Model (VLM) represents an avenue for future work.

- **GNN Constraints:** Transitioning to Graph Attention Networks (GATs) for structural modeling represents a theoretically robust paradigm. However, training PyTorch Geometric models incurs heavy memory overhead during scatter/gather operations, rendering it computationally demanding for consumer edge devices.

## VIII. CONCLUSION

By fusing context-aware structural HTML sequences, isolated semantic JavaScript intent via learned attention pooling, and expanded lexical heuristics, the proposed Dual-Modal Stacking Ensemble successfully identifies highly evasive zero-day phishing pages. This decoupled architecture demonstrates that focusing on intrinsic functional routing yields significantly higher precision, eliminates Domain Blindness, and remains computationally viable on strictly constrained hardware profiles.

## REFERENCES

- [1] Z. Feng et al., “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [3] A. Oest et al., “Sunrise to Sunset: Exploring the Lifespan and Predictability of Phishing URLs,” in *Proceedings of the Internet Measurement Conference*, 2020, pp. 134–147.
- [4] J. Zhang et al., “Phishing the Phishers with SpecularNet: Hierarchical Graph Autoencoding for Reference-Free Web Phishing Detection,” *arXiv preprint arXiv:2603.01874*, 2026.
- [5] M. A. Adebowale, K. T. Lwin, E. Sánchez, and M. A. Hosseini, “Intelligent web-phishing detection and protection scheme using integrated features of Images, frames and text,” *Expert Systems with Applications*, vol. 115, pp. 300–313, 2019.
- [6] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, “Machine learning based phishing detection from URLs,” *Expert Systems with Applications*, vol. 117, pp. 345–357, 2019.
- [7] C. Opara, Y. Wei, and Y. Chen, “HTMLPhish: Enabling phishing web page detection by applying deep learning techniques on HTML analysis,” in *2020 International Joint Conference on Neural Networks (IJCNN)*, IEEE, 2020, pp. 1–8.
- [8] J. Lin et al., “Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 3793–3810.
- [9] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [11] R. Sommesse et al., “PhishCatch: A Phishing Detection Tool,” in *Proceedings of the 2022 ACM on Internet Measurement Conference*, 2022, pp. 1–14.